

Detailed Methodological Guide

Portfolio Optimization with Machine Learning, Market Regimes and Quantitative Methods

Jaime

Internal working document (not deliverable)

Contents

1	General project objective	4
2	Technological environment and libraries	4
3	Market, asset universe and time horizon	5
3.1	Market and benchmark	5
3.2	Asset universe	5
3.3	Time horizon	5
4	Data: download, cleaning and structure	5
4.1	Data download	5
4.2	Cleaning and alignment	6
4.3	Data structure	6
5	Construction of the monthly target	6
5.1	Target definition	6
5.2	Implementation	7
6	Construction of the monthly dataset	7
6.1	Time windows	7
6.2	Final dataset structure	7
7	Final features for the monthly model	7
7.1	Past returns	8
7.2	Volatility	8

7.3	Trend (moving averages)	8
7.4	Volume-based features	8
7.5	OHLC-based features (derived, not raw)	8
7.6	Optional technical indicators	9
8	General pipeline and rolling window scheme	9
9	Predictive models	10
9.1	Shallow models (classical ML)	10
9.2	Deep models (PyTorch)	10
9.2.1	Small LSTM	10
9.2.2	Medium LSTM	10
9.2.3	Simple CNN	11
9.2.4	Medium CNN	11
9.3	Training naive models	11
10	Normalization and data preparation	11
11	Market Regime Detection through Clustering with Wasserstein Distance	12
11.1	Motivation	12
11.2	Construction of return distributions	12
11.3	Wasserstein distance between windows	13
11.4	Unsupervised clustering	13
11.5	Economic interpretation of regimes	13
11.6	Assignment of regimes over time	14
11.7	Integration into the pipeline	14
12	Model selection by regime	14
12.1	Main method: selection of the best naive model per regime	15
12.2	Optional extension: retrain specific models per regime	15
13	Portfolio optimization	16
13.1	General approach	16
13.2	Implementation with <code>riskfolio-lib</code>	16
14	Robust Monte Carlo Simulation	17
14.1	Motivation and theoretical foundation	17
14.2	Probabilistic model for future returns	17
14.3	Scenario generation	18

14.4	Optimization per scenario	18
14.5	Robust weight aggregation	19
14.5.1	Mean of weights	19
14.5.2	Median of weights	19
14.5.3	Conservative percentile	19
14.6	Financial interpretation	19
14.7	Relationship with decision theory under uncertainty	20
14.8	Conclusion	20
15	Transaction costs	20
16	Evaluation	20
16.1	Basic statistical evaluation	20
16.2	Advanced statistical evaluation: significance tests	21
16.2.1	Tests for linear regression (Statsmodels)	21
16.2.2	Tests to compare models	21
16.2.3	Tests on predictive utility	21
16.3	Financial evaluation	22
17	Optional extension: intraday modeling with Transformers	22
17.1	Intraday data	22
17.2	Sequence construction	22
17.3	Prediction task	22
17.4	Models	22
17.5	Metrics	23
18	Closing of the methodological design	23

1 General project objective

The objective of this project is to design and document a complete quantitative pipeline to:

- Predict monthly returns of a set of S&P 500 stocks using Machine Learning (ML) and Deep Learning (DL) models.
- Detect market regimes and adapt model selection to each regime.
- Optimize portfolios robustly, incorporating uncertainty through Monte Carlo simulation and transaction costs.
- Evaluate the performance of the resulting portfolios against classical benchmarks.

This document is a closed technical guide: it defines what will be done, how it will be done, with which libraries and with what final design. It is not the thesis dissertation, but rather the methodological foundation upon which it will be built.

2 Technological environment and libraries

The project will be implemented in Python. The base environment will be:

- **Python version:** 3.10+ (ideally 3.10 or 3.11).
- **Environment manager:** conda or venv.

Main libraries:

- **Data manipulation:** pandas, numpy.
- **Data download:** yfinance.
- **Classical Machine Learning:** scikit-learn.
- **Gradient boosting:** xgboost.
- **Deep Learning:** pytorch (and torchvision if necessary).
- **Portfolio optimization:** riskfolio-lib.
- **Financial metrics:** empyrical.
- **Wasserstein distance calculation:** scipy.stats.

- **Technical indicators (optional):** `ta` or manual implementation.

The code will be organized into modules (for example, `data_processing.py`, `features.py`, `models.py`, `regimes.py`, `optimization.py`, `backtest.py`) to maintain clarity and reusability.

3 Market, asset universe and time horizon

3.1 Market and benchmark

- **Market:** United States.
- **Main benchmark:** S&P 500 (index).

3.2 Asset universe

- **Universe:** 30 large-cap stocks belonging to the S&P 500.
- **Selection criteria:**
 - High liquidity.
 - Sector diversity (technology, financial, consumer, healthcare, energy, industrial, etc.).
 - Sufficient history in the study period.

3.3 Time horizon

- **Gross data period:** 2010–2026 (daily).
- **Effective backtesting period:** 2013–2026.

The first years are used to build sufficient historical windows for features and regimes.

4 Data: download, cleaning and structure

4.1 Data download

`yfinance` will be used to download daily data:

- Adjusted close price (`Adj Close`).

- Volume.
- OHLC (Open, High, Low, Close) for intraday extension and possible derived features.

4.2 Cleaning and alignment

- Align all assets on the same time index.
- Fill holidays and non-trading days with NaN and avoid forward-fill in prices.
- Remove assets with excessive gaps in the study period.

4.3 Data structure

We will work with:

- **Daily price DataFrame:** rows = dates, columns = assets.
- **Daily volume DataFrame:** same structure.
- **Daily returns DataFrame:** calculated as log-returns.

5 Construction of the monthly target

The objective is to predict the monthly log-return of the next month for each asset.

5.1 Target definition

Let $P_t^{(i)}$ be the adjusted close price of asset i on day t . We define the monthly log-return of asset i between the start of month m and the end of month m as:

$$r_m^{(i)} = \ln \left(\frac{P_{\text{end of month } m}^{(i)}}{P_{\text{start of month } m}^{(i)}} \right)$$

The target to predict on the last day of month m will be:

$$y_{m+1}^{(i)} = r_{m+1}^{(i)}$$

5.2 Implementation

- Use `resample('M')` to identify the last day of each month.
- For each asset, obtain the price of the first and last day of each month.
- Calculate the monthly log-return.
- Shift the target one month forward to associate it with the previous month's features.

6 Construction of the monthly dataset

Although the data is daily, the model works at the monthly level. For each month m and asset i :

- Features are calculated using data up to the last day of month m .
- The target $y_{m+1}^{(i)}$ is associated.

6.1 Time windows

Rolling windows over daily data will be used to construct features:

- 1 month $\approx$ 21 days.
- 3 months $\approx$ 63 days.
- 6 months $\approx$ 126 days.

6.2 Final dataset structure

Each row of the monthly dataset corresponds to a pair (asset i , month m) and contains:

- Features calculated with data up to the end of month m .
- Target $y_{m+1}^{(i)}$.

7 Final features for the monthly model

The feature design seeks a balance between:

- Sufficient information to capture patterns.
- Simplicity and robustness (avoid overfitting).
- Reasonable interpretability.

7.1 Past returns

- 1-month log-return:

$$r_{1m}^{(i)}(m) = \ln \left(\frac{P_{\text{end of month } m}^{(i)}}{P_{\text{end of month } m-1}^{(i)}} \right)$$

- 3-month cumulative log-return.
- 6-month cumulative log-return.

7.2 Volatility

- 1-month volatility: standard deviation of daily log-returns of the last 21 days.
- 3-month volatility: standard deviation of the last 63 days.

7.3 Trend (moving averages)

- MA20: 20-day moving average of the closing price.
- MA60: 60-day moving average.
- MA20/MA60 ratio: relative trend indicator.

7.4 Volume-based features

- 20-day average volume.
- 60-day average volume.
- Current volume / 20-day average volume ratio.

7.5 OHLC-based features (derived, not raw)

Although the monthly pipeline does not need raw OHLC, it is useful to derive some features:

- Average intraday range of the month:

$$\text{Range}_{\text{avg}}(m) = \text{mean}_{t \in \text{month } m}(\text{High}_t - \text{Low}_t)$$

- Average candle body:

$$\text{Body}_{\text{avg}}(m) = \text{mean}_{t \in \text{month } m} |\text{Close}_t - \text{Open}_t|$$

- Average True Range (to capture gaps).

These features are calculated with **pandas** and, if desired, with the help of libraries like **ta**.

7.6 Optional technical indicators

Optional, but useful if time permits:

- RSI 14 days.
- ATR (Average True Range) 14 days.

8 General pipeline and rolling window scheme

The pipeline is executed in a monthly *rolling* scheme. For each month m in the backtesting period:

1. Construct the dataset with data up to the end of month m (historical features and targets).
2. Fit the scaling (**StandardScaler**) with data up to m .
3. Train “naive” models (without regimes) with data up to m .
4. Evaluate and save results of the naive models.
5. Detect the market regime in month m .
6. Select the appropriate model according to the regime (main method).
7. Generate return predictions for month $m + 1$.
8. Generate Monte Carlo scenarios.
9. Optimize the portfolio for month $m + 1$.
10. Apply transaction costs.

11. Record the net return of month $m + 1$.

This process is repeated month by month, without using future information (no *data leakage*).

9 Predictive models

9.1 Shallow models (classical ML)

The following models from `scikit-learn` and `xgboost` will be used:

- Linear regression (`LinearRegression`).
- Ridge, Lasso, Elastic Net.
- Random Forest (`RandomForestRegressor`).
- XGBoost (`XGBRegressor`).
- Simple MLP (`MLPRegressor`, optional).

Hyperparameters will be set reasonably (no extreme search will be done to avoid overfitting and excessive time consumption).

9.2 Deep models (PyTorch)

Simple but representative models will be used:

9.2.1 Small LSTM

- LSTM layer with 32 units.
- Dropout 0.2.
- Final dense layer with 1 neuron (scalar output).

9.2.2 Medium LSTM

- LSTM layer with 64 units.
- Dropout 0.3.
- Intermediate dense layer of 32 neurons.
- Final dense layer of 1 neuron.

9.2.3 Simple CNN

- Conv1D(32 filters, kernel=3).
- MaxPool1D.
- Flatten.
- Dense(32).
- Dense(1).

9.2.4 Medium CNN

- Conv1D(32, kernel=3).
- Conv1D(16, kernel=3).
- MaxPool1D.
- Flatten.
- Dense(32).
- Dense(1).

9.3 Training naive models

In the first phase, all models are trained without distinguishing market regimes. This provides:

- A clear baseline.
- Comparable results between models.
- Data to analyze which models work better in which environments.

10 Normalization and data preparation

`StandardScaler` from `scikit-learn` will be used:

- Fitted only with data available up to month m .
- Applied to all numerical features.

For deep models, PyTorch tensors will be used, maintaining the same scaling.

11 Market Regime Detection through Clustering with Wasserstein Distance

Market regime detection is a central component of the pipeline, as it allows adapting the choice of predictive model to the statistical environment in which the market finds itself. In this project, regime detection is not based on predefined rules or technical indicators, but on an unsupervised approach that identifies regimes directly from the statistical structure of returns. To do this, a clustering method based on the Wasserstein distance between return distributions is used, following the methodology proposed in recent regime analysis literature.

11.1 Motivation

Financial markets exhibit dynamics that change over time: periods of stability, phases of high volatility, episodes of sharp declines, etc. These changes are not always evident from simple indicators, and may be better reflected in the complete shape of the return distribution than in point metrics such as mean or volatility.

The use of Wasserstein distance allows comparing complete return distributions, capturing differences in:

- mean,
- volatility,
- skewness,
- kurtosis (heavy tails),
- general distribution shape.

This allows detecting market regimes in a richer and more robust way than with methods based solely on technical indicators.

11.2 Construction of return distributions

The procedure begins by dividing the market index history (S&P 500 or SPY) into fixed-length rolling windows, typically 60 days. For each window w , the empirical distribution of daily returns is obtained:

$$\mathcal{D}_w = \{r_t, r_{t+1}, \dots, r_{t+59}\}$$

Each window represents a local state of the market.

11.3 Wasserstein distance between windows

For each pair of windows (i, j) the Wasserstein-2 distance between their empirical distributions is calculated:

$$d_{ij} = W_2(\mathcal{D}_i, \mathcal{D}_j)$$

The Wasserstein distance measures the "minimum cost" of transforming one distribution into another, capturing differences in the entire shape of the distribution, not just in specific moments.

This produces a distance matrix:

$$D = [d_{ij}] \in \mathbb{R}^{T \times T}$$

where T is the number of windows.

11.4 Unsupervised clustering

From the distance matrix D , an unsupervised clustering method is applied to identify groups of windows with similar distributions. Hierarchical clustering or k-means is applied on an embedding obtained through techniques such as MDS (Multidimensional Scaling).

The number of clusters is set to $k = 3$, following the literature and the standard financial interpretation of three main regimes.

The result is a set of clusters:

$$C_1, C_2, C_3$$

Each cluster represents a distinct statistical regime of the market.

11.5 Economic interpretation of regimes

Although regimes are obtained in an unsupervised manner, it is possible to interpret them economically by analyzing the average characteristics of the windows belonging to each cluster:

- average return,
- average volatility,
- skewness,
- kurtosis.

This allows mapping clusters to classical financial regimes:

- **Regime 1 (Bull Market):** narrow distributions, low or moderate volatility, positive returns.
- **Regime 2 (Bear Market):** wide distributions, high volatility, heavy tails, negative returns.
- **Regime 3 (Sideways / Mean-Reverting):** returns close to zero, intermediate volatility, stable distribution shape.

This mapping allows maintaining financial interpretability without giving up the power of the unsupervised approach.

11.6 Assignment of regimes over time

Each window w receives a regime label according to the cluster to which it belongs. The label of month m is assigned according to the window ending in that month.

This produces a time series of regimes:

$$R_m \in \{1, 2, 3\}$$

which is subsequently used to select the most appropriate predictive model for each environment.

11.7 Integration into the pipeline

The regime detected in month m determines which model will be used to predict the returns of month $m + 1$. For each regime, the model that has historically shown better performance within that regime is selected, according to the evaluation metrics defined in the project.

This approach allows dynamically adapting the predictive model to market conditions, improving the robustness and stability of the pipeline.

12 Model selection by regime

Once regimes are defined, they are used to adapt model selection.

12.1 Main method: selection of the best naive model per regime

Idea

Models are not retrained by regime. Instead:

- All naive models are trained with all data (up to each time point).
- It is analyzed, a posteriori, which model works best in each regime.
- In real time, the regime is detected and the model that has historically worked best in that regime is chosen.

Procedure

1. For each historical month m :
 - The regime R_m is known.
 - The predictions of each model and the actual returns are known.
2. For each regime k and model j :
 - Filter months with $R_m = k$.
 - Calculate metrics:
 - MAE, RMSE.
 - Prediction–actual correlation.
 - Sign accuracy %.
 - Optional: Sharpe of the strategy based on that model.
3. For each regime k , select the model j^* with the best performance.
4. In the rolling backtest:
 - In month m , detect the regime R_m .
 - Use the model $j^*(R_m)$ to predict returns of month $m + 1$.

12.2 Optional extension: retrain specific models per regime

As an advanced extension:

- For each regime k , a specific model is trained using only data from that regime.
- In real time, the regime is detected and the model trained for that regime is used.

Risks:

- Few data per regime.
- Higher risk of overfitting.
- Higher computational cost.

Therefore, it is kept as an optional extension, not as the core.

13 Portfolio optimization

13.1 General approach

Given a vector of expected returns $\hat{r}$ (predictions) and a covariance matrix Σ , we seek a weight vector w that maximizes the expected return subject to constraints.

$$\max_w w^\top \hat{r}$$

Subject to:

- $w_i \geq 0$ (no shorts).
- $\sum_i w_i = 1$ (fully invested).
- $w_i \leq w_{\max}$ (for example, 0.10–0.15).

13.2 Implementation with riskfolio-lib

- Build a portfolio object with historical and/or predicted returns.
- Specify the risk model (classical covariance or Ledoit–Wolf).
- Define weight constraints.
- Solve the optimization problem.

Both a return maximization approach for a given risk level and mean-variance variants can be used.

14 Robust Monte Carlo Simulation

Monte Carlo simulation constitutes an essential component of the pipeline, as it allows explicitly incorporating the inherent uncertainty in both return predictions and the estimation of the covariance matrix. Instead of optimizing the portfolio solely on a single expected return vector $\hat{r}$, multiple plausible scenarios of future returns are generated and the portfolio is optimized in each of them. This approach produces more stable, diversified portfolios that are less sensitive to estimation error, mitigating one of the classical problems of mean-variance optimization.

14.1 Motivation and theoretical foundation

Classical Markowitz optimization uses a single pair $(\hat{r}, \Sigma)$, where:

$$\hat{r} \in \mathbb{R}^N \quad \text{and} \quad \Sigma \in \mathbb{R}^{N \times N}$$

represent, respectively, the estimated expected returns and covariance matrix. However:

- $\hat{r}$ is a prediction subject to error.
- Σ is a noisy estimate based on a finite sample.
- Classical optimization is extremely sensitive to small perturbations in $\hat{r}$.

Therefore, optimizing over a single estimated point can lead to unstable and unrealistic solutions. Monte Carlo allows approximating the distribution of possible futures and optimizing over it.

14.2 Probabilistic model for future returns

It is assumed that future returns follow a multivariate normal distribution:

$$r \sim \mathcal{N}(\hat{r}, \Sigma)$$

This choice is justified by:

- its standard use in quantitative finance,
- its ability to capture correlations between assets,
- its computational efficiency,
- its compatibility with convex optimization.

Although the actual return distribution is not normal, this approximation is sufficient to generate plausible scenarios consistent with the correlation structure.

14.3 Scenario generation

Let S be the number of Monte Carlo scenarios (typically $S = 500$ or $S = 1000$). Each scenario is generated as:

$$r^{(s)} = \hat{r} + Lz^{(s)}, \quad z^{(s)} \sim \mathcal{N}(0, I)$$

where L is the Cholesky decomposition of Σ :

$$\Sigma = LL^\top$$

This procedure guarantees that:

- scenarios respect correlations between assets,
- the dispersion of returns is consistent with estimated volatility,
- a realistic range of possible futures is explored.

14.4 Optimization per scenario

For each scenario $r^{(s)}$ the problem is solved:

$$\max_{w^{(s)}} (w^{(s)})^\top r^{(s)}$$

subject to:

$$w_i^{(s)} \geq 0, \quad \sum_i w_i^{(s)} = 1, \quad w_i^{(s)} \leq w_{\max}$$

where $w_{\max}$ is typically 0.10–0.15.

This process produces a set of S weight vectors:

$$\mathcal{W} = \{w^{(1)}, w^{(2)}, \dots, w^{(S)}\}$$

Each $w^{(s)}$ represents the optimal portfolio under a possible future consistent with the available information.

14.5 Robust weight aggregation

The objective is to obtain a single robust weight vector w^* that synthesizes the information contained in the S scenarios.

14.5.1 Mean of weights

$$w^* = \frac{1}{S} \sum_{s=1}^S w^{(s)}$$

Produces smooth and diversified portfolios. It is suitable when a conservative bias is not desired.

14.5.2 Median of weights

$$w_i^* = \text{median} \left(w_i^{(1)}, \dots, w_i^{(S)} \right)$$

More robust against extreme scenarios or degenerate solutions.

14.5.3 Conservative percentile

For a level α (for example, 25%):

$$w_i^* = \text{percentile}_\alpha \left(w_i^{(1)}, \dots, w_i^{(S)} \right)$$

Favors prudent portfolios, reducing exposure to assets with high uncertainty.

14.6 Financial interpretation

The Monte Carlo approach allows:

- explicitly incorporating uncertainty in predictions,
- avoiding extreme solutions typical of classical optimization,
- obtaining more stable portfolios over time,
- capturing the dispersion of possible futures,
- reducing sensitivity to errors in $\hat{r}$ and Σ ,
- improving robustness against regime changes.

In essence, the optimization over a single estimated future is replaced by an optimization over a range of plausible futures, which produces more realistic decisions consistent with the uncertain nature of financial markets.

14.7 Relationship with decision theory under uncertainty

The method can be interpreted as:

- a numerical approximation to stochastic optimization,
- a particular case of *robust optimization*,
- a way to smooth the objective function through integration over the return distribution,
- a technique to approximate the expected value of the optimal policy under uncertainty.

In formal terms, the implicit objective is:

$$w^* = \arg \max_w \mathbb{E}_{r \sim \mathcal{N}(\hat{r}, \Sigma)}[w^\top r]$$

but subject to additional stability and robustness constraints through aggregation.

14.8 Conclusion

Robust Monte Carlo simulation is a fundamental component of the pipeline, as it allows obtaining more stable, diversified portfolios that are resistant to estimation error. Its integration with classical optimization provides an appropriate balance between mathematical sophistication, interpretability and computational feasibility.

15 Transaction costs

They are modeled as proportional to the rebalancing volume:

$$\text{Cost}_m = c \cdot \sum_i |w_{m,i} - w_{m-1,i}|$$

where c is the cost per unit of weight change.

The net return of month m is:

$$R_{\text{net},m} = R_{\text{gross},m} - \text{Cost}_m$$

16 Evaluation

16.1 Basic statistical evaluation

MAE, RMSE, correlation, sign accuracy.

16.2 Advanced statistical evaluation: significance tests

In addition to basic metrics, statistical tests will be applied to rigorously evaluate the quality of models and the significance of their parameters.

16.2.1 Tests for linear regression (Statsmodels)

Using `statsmodels.api.OLS` the following are obtained:

- **t-tests** for individual coefficients:

$$t_j = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)}$$

They allow evaluating whether each feature has a significant effect on the return.

- **p-values** associated with each coefficient.
- **F-test** of global model significance.
- **Durbin–Watson** for residual autocorrelation.
- **Breusch–Pagan** for heteroskedasticity.
- **Jarque–Bera** for residual normality.

16.2.2 Tests to compare models

- **Diebold–Mariano test** to compare prediction errors between two models.
- **Mean difference test** for MAE or RMSE.
- **Binomial test** to evaluate whether sign accuracy is greater than 50%.

16.2.3 Tests on predictive utility

- Test of mean errors = 0.
- Test of prediction–actual correlation $\neq 0$.

These tests allow evaluating whether predictions contain statistically significant information.

16.3 Financial evaluation

Sharpe, volatility, annualized return, drawdown, turnover.

Benchmarks: 1/N, Buy & Hold, Markowitz without ML.

17 Optional extension: intraday modeling with Transformers

This section is an independent extension of the main pipeline. It does not affect the monthly design, but serves as an exploration of modern architectures.

17.1 Intraday data

- Asset: SPY (S&P 500 ETF).
- Frequency: 5 minutes.
- Period: 2020–2024.
- Variables: OHLCV (Open, High, Low, Close, Volume).

17.2 Sequence construction

- Sliding windows of length 100 (500 minutes $\approx$ 1–2 trading days).
- Each window is represented as a sequence of normalized OHLCV vectors.

17.3 Prediction task

Possible examples:

- Binary classification: up/down in the next interval.
- Multi-class classification: return range.

17.4 Models

- Small LSTM (baseline).
- Small 1D Transformer:
 - Linear embedding of the input.

- 2–3 multi-head attention layers.
- Final dense layer.

17.5 Metrics

- Accuracy.
- F1-score.
- AUC-ROC.

The objective of this extension is exploratory: to compare the performance of Transformers versus LSTM in an intraday environment.

18 Closing of the methodological design

The final project design is closed with the following key elements:

- Monthly return prediction with ML and DL models (naive first).
- Market regime detection with simple methods (and Wasserstein as an extension).
- Dynamic model selection by regime (main method: choose the best naive model per regime).
- Portfolio optimization with `riskfolio-lib`.
- Robustness through Monte Carlo.
- Explicit transaction costs.
- Complete statistical and financial evaluation.
- Optional intraday extension with Transformers and OHLCV.

This document defines in a concrete and closed way what will be done and how it will be done, serving as a master guide for implementation and, subsequently, for writing the thesis.